

Felhasználói dokumentáció

A generikus_graf.exe program segítségével irányítatlan generikus gráfokat tudunk kezelni. A program képes egy mátrixszal leírt állományból a gráfot felépíteni, majd azt bármelyik csúcsából bejárni, szélességi bejárást végezve. A szélességi bejárásokat elvégezve el tudja dönteni, hogy a gráf összefüggő-e.

A program futtatása

A program meghívása a parancssorból a következőképpen néz ki:

```
generikus_graf.exe [GRAF_FILE_PATH]
```

Az első paraméterben a gráfot leíró mátrix fájljának az elérési útját kell megadni relatív útvonallal. Amennyiben paraméter nélkül hívjuk meg a programot, a program lefuttatja a definiált teszteket, amelyek ellenőrzik a program helyes működését. Ehhez a tesztfájloknak a végrehajtható állománnyal egy könyvtárban kell lenniük.

Például:

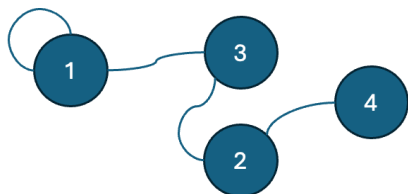
```
generikus_graf.exe matrix_2komponens_6csucs.txt
```

Gráfleírás

A gráfot egy szomszédsági mátrixszal adhatjuk meg. Ha két csúcs között van él, akkor azt egy 1-es, ha nincs, akkor azt egy 0 jelöli. Mivel a gráf irányítatlan, ha az 1-es csúcs össze van kötve a 2-essel, akkor a 2-esnek is össze kell lennie kötve az 1-essel, vagyis a mátrixnak szimmetrikusnak kell lennie. Az alábbi példa egy 4 csomópontú gráf leírását tartalmazza:

1	0	1	0
0	0	1	1
1	1	0	0
0	1	0	0

A fenti példa az alábbi gráfot írja le.



Eredmény

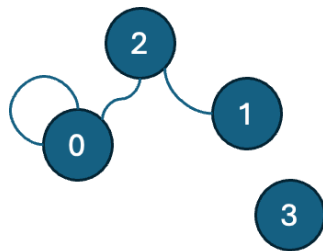
A futtatás során a program szélességi bejárást végez a gráfon, és eldönti, hogy összefüggő-e a gráf, megszámlálja az éleit és a csúcsait. Ha a gráf nem összefüggő, akkor kiírja, hogy az egyes komponensek melyik csúcsokból állnak.

Az alábbi példa a fenti összefüggő gráf esetén mutatja a kimenetet

```
Parancssorból meghívott matrix: matrix_4csucs_osszefuggo_hurokel.txt  
Csomópontok száma: 4  
Élek száma: 4  
A graf összefuggo
```

Egy nem összefüggő gráf esetén:

1	0	1	0
0	0	1	0
1	1	0	0
0	0	0	0



a futtatás eredménye a következő:

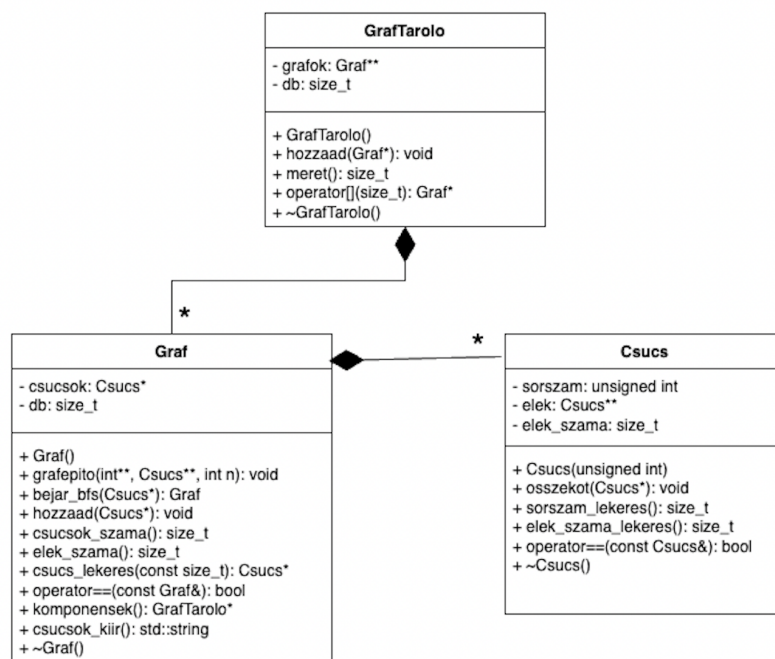
```
Parancssorból meghívott matrix: matrix_4csucs_2komponens_hurokel.txt  
A graf nem összefuggo  
Csomópontok száma: 4  
Élek száma: 3  
1. komponens: 0, 2, 1  
2. komponens: 3
```

Programozói dokumentáció

A programnak lehetővé kell tennie, hogy egy fájlból egy generikus irányítatlan gráfot olvasson be, és azon szélességi bejárást végezzen. A fájlban a gráfot egy szomszédsági mátrixszal kell megadni, aminek pontos struktúráját a specifikáció tartalmazza.

A program a fájlból a mátrixot először egy char tömbbe olvassa be, majd ebből a char tömbből épít egy $n \times n$ -es int tömböt, ami reprezentálja a szomszédsági mátrixot. A mátrix építése során ellenőrzéseket végez (pl. négyzetes-e a mátrix, csak 1-esek és 0-k vannak benne, stb.).

A beolvasott gráf elemzéséhez az alábbi osztály-hierarchiát definiáltuk:



A Csucs a gráf egy csúcsát reprezentálja. A csúcsot a sorszámaival azonosítjuk. A Graf Csucsok gyűjteménye. A Csucsok összekötése a Csucs osztályban található *elek* tömbben tárolódik. A GrafTarolo egy tároló, amiben Graf objektumokat tudunk tárolni. Ezt arra használjuk, hogy az összefüggőség eldöntéséhez szükséges részgráfokat eltároljuk.

Csucs osztály

Tagváltozók

unsigned int sorszam;

a Csucs azonosítója

Csucs elek;**

Csucsból kifutó élek pontjainak tömbje

size_t elek_szama;

Csucsból kifutó élek száma

Függvények

Csucs(unsigned int sorszam = 0)

A konstruktor a bemeneti paraméterként megadott azonosító sorszámmal hoz létre egy csúcsot.

void osszekot(Csucs* v)

összeköti az adott Csúcsot a paraméterként megkapott Csúccsal és viszont (irányítatlan élék)

size_t sorszam_lekeres() const

getter a sorszamra

size_t elek_szama_lekeres() const

getter az élék számára

Csucs* el_lekeres (size_t index) const

azt a csúcsot adja vissza, amelyikre az indexedik él mutat

bool van_hurokel() const;

meghatározza, hogy az adott csúcsra illeszkedik-e hurokél

bool operator==(const Csucs &csucs)

két Csucs egyenlő, ha a sorszámuk azonos, és a belőle kifutó élék ugyanoda mutatnak

~Csucs()

A destruktork felszabadítja a dinamikusan lefoglalt *elek* tömböt

Graf osztály

Tagváltozók

Csucs csucsok**

a Grafhoz tartozó Csucsok pointereinek tömbje

size_t db

Csucsok száma

Függvények

Graf()

A konstruktor egy üres gráfot hoz létre

void grafepito(int matrix, Csucs** graf_csucsai, int n)**

Felépíti a gráfot a megadott Csucsokból és a szomszédsági mátrixból, n a szomszédsági mátrix sorainak és oszlopainak a száma

Graf* bejar_bfs (Csucs* v)

BFS algoritmussal bejárja a Graf egy komponensét és ezt az új Grafot adja vissza, a Csucsokat nem másolja, csak mutatókat tárol rájuk

void hozzaad(Csucs* v)

új Csucs hozzáadása a Grafhoz

size_t csucsok_szama() const

getter a db változóra

size_t elek_szama()

megadja a gráf éleinek számát

Csucs* csucs_lekeres (const size_t index) const

visszaadja az indexedik csúcsát a gráfnak

bool operator==(const Graf &graf)

két Graf akkor egyenlő, ha azonos Csucsokat tartalmaz (Csucsok alapján azonos éleket)

GrafTarolo* komponensek()

bejárja a Grafot minden Csucsból kiindulva, és a különböző komponenseket (Graf) egy GrafTaroloban adja vissza

std::string csucsok_kiir()

kiírja a Csucsok sorszámát vesszővel elválasztva

~Graf()

Felszabadítja a dinamikusan lefoglalt Csucs objektumokra mutató pointereket.

GrafTarolo osztály

Tagváltozók

Graf grafok**

Grafokra mutató pointerek tömbje

size_t db

a tárolóban tárolt Grafok száma

Függvények

GrafTarolo()

Létrehoz egy üres tárolót

void hozzáad (Graf* graf)

egy Gráf pointerét hozzáadja a tárolóhoz, ha még nincs benne

size_t meret() const

getter a db-ra

Graf* operator[] (size_t index)

visszaadja az indexedik Grafra mutató pointert

~GrafTarolo()

Felszabadítja a dinamikusan lefoglalt Graf objektumokra mutató pointereket.

beolvaso.cpp

Segédfüggvényeket tartalmaz a szomszédsági mátrix fájlból való beolvasásához

char* fajlbeolvaso(std::string fajlnev)

egy karaktertömbbe beolvassa a fájl tartalmát

int matrixba_olvas(std::string fajlnev, int& n)**

a karaktertömbből egy nxn-es int tömböt készít

main.cpp

A program futása során a segédfüggvények segítségével beolvassa a fájlból a szomszédsági mátrixot. Ezután létrehoz egy példányt a Graf objektumból és felépíti a gráfot a Graf.grafepito() függvény segítségével. Ezután lekéri ennek a komponenseit, ami egy GrafTaroloban visszaadja az összes komponenst. A komponensek száma alapján eldönti, hogy összefüggő-e a gráf, és a Graf.tagfüggvényei segítségével kiírja a részgráfok adatait.